Лабораторная работа №9.
Применение регулярных выражений JavaScript при обработке Web-форм

Цель работы: научиться использовать простейшие элементы регулярных выражений в JavaScript для поиска подстрок, структура и содержание которых описывается нетривиальным шаблоном.

Теоретический материал

Регулярные выражения – система поиска текстовых фрагментов в электронных документах, основанная на специальной системе записи образцов для поиска.

Образец, задающий правило поиска, называется «*шаблоном*». Применение регулярных выражений принципиально преобразило технологии электронной обработки текстов.

С помощью регулярных выражений можно задавать структуру искомого шаблона и его позицию внутри строки (например, в начале или в конце строки, на границе или не на границе слова).

При описании структуры шаблона используются:

- гибкая система квантификаторов (операторов повторения);
- операторы описания наборов символов и их типа (числовые, нечисловые, специальные).

Для того, чтобы задать положение искомого фрагмента внутри строки, можно использовать один из следующих операторов:

Представление	Позиция
^	Начало строки
$	Конец строки
\b	Граница слова
\B	Не граница слова
(?=шаблон)	Искомая строка следует после указанной строкой (с просмотром вперед)
(?!=шаблон)	Искомая строка не следует после указанной строки (с просмотром вперед)
(?<=шаблон)	Искомая строка следует после указанной строкой (с просмотром назад)
(?<!шаблон)	Искомая строка не следует после указанной строки (с просмотром назад)

Кроме того, язык регулярных выражений предоставляет набор квантификаторов, позволяющих указать число повторений шаблона:

Представление	Число повторений
$\{n\}$	Ровно n
$\{m, n\}$	От m до n включительно
$\{m,\}$	Не менее m
$\{,n\}$	Не более n

Имеются и более простые квантификаторы:

Представление	Число повторений	Эквивалент
*	Ноль или более	$\{0,\}$
+	Одно или более	$\{1,\}$
?	Ноль или одно	$\{0,1\}$

Для задания внутри шаблона группы символом можно использовать следующие операторы:

Оператор	Описание
[xyz]	Любой символ из указанного множества
[^xyz]	Любой символ не входящий в указанное множество
[x-z]	Любой символ из указанного диапазона
[^x-z]	Любой символ не входящий в указанный диапазон
. (*точка*)	Любой символ кроме символов разрыва или переноса строки
\w	Любой буквенно-цифровой символ, включая символ подчеркивания
\W	Любой не буквенный символ
\d	Любая цифра
\D	Любой нецифровой символ
\s	Люой не отображаемый символ
\S	Любой символ (кроме неотображаемых символов)

Для группировки отдельных частей шаблона можно использовать следующие операторы:

Оператор	Описание
( )	Поиск группы символов внутри скобок и сохранение найденного соответствия
(?:)	Поиск группы символов внутри скобок без сохранения найденного соответствия
|	Комбинирование частей в одно выражения с последующим поиском любой из частей в отдельности. Аналогично оператору «ИЛИ».

Если шаблон поиска включает специальные (как правило неотображаемые) символы, для их описания можно использовать следующие обозначения:

Обозначение	Описание
\0	Символ с нулевым кодом
\n	Символ новой строки

\r	Символ начала строки
\t	Символ табуляции
\v	Символ вертикальной табуляции
\xxx	Символ, имеющий заданный восьмеричный ASCII код *xxx*
\xdd	Символ, имеющий заданный шестнадцатиричный ASCII код *dd*
\uxxxx	Символ, имеющий ASCII код выраженный ЮНИКОДОМ *xxxx*

Квантификаторам в регулярных выражениях соответствует максимально длинная строка из возможных (т.е. квантификаторы являются «жадными»). Это может приводить к некоторым проблемам. Например, шаблон (**<.*>**) описывающий на первый взгляд теги HTML на самом деле будет выделять более крупные фрагменты в документе.

Например, строка вида

* <i>Регулярные выражения<i> - удобный инструмент для поиска в строках *

формально соответствует указанному выше шаблону

Для решения данной проблемы можно использовать два подхода.

1. В регулярном выражении учитываются символы, не соответствующие желаемому образцу (например, **<[^>]*>** для вышеописанного случая).

2. Определение квантификатора как *нежадного* (ленивого) - большинство реализаций позволяют это сделать, добавив после него знак вопроса.

Например, по шаблону (<.*?>) будут найдены все теги из рассмотренной строки.

Таким образом, получаются следующие «нежадные» модификации квантификаторов:

Квантификатор	**Описание**
*?	«не жадный» эквивалент *
+?	«не жадный» эквивалент +
{n,}?	«не жадный» эквивалент {n,}

Следует, однако, иметь в виду, что использование «ленивых» квантификаторов может привести к ситуации, когда выражению соответствует слишком короткая, в частности, пустая строка.

Использование регулярных выражений в JavaScript

При поиске по тексту можно использовать шаблон, описывающий подстроку. В JavaScript такой шаблон может быть описан с помощью объекта *RegExp*. В простейшем случае такой шаблон описывает отдельный символ, однако имеет смысл его использовать для регулярных выражений.

Следующий ниже код описывает *RegExp* объект с именем *pattn*, содержащий регулярное выражение, описывающее целое десятичное число:

var *pattn* = new RegExp("/[0-9]+/");

Объект *RegExp* имеет 3 встроенных метода: *test*(), *exec*() и *compile*().

Метод *test()* выполняет поиск по шаблону:

```
var pattn = new RegExp("[0-9]+");
document.write(pattn.test("38 попугаев"));
```

В результате будет получено true.

Метод *exec*() выполняет поиск подстроки по шаблону и возвращает найденные соответствия; если соответствий нет, возвращается значение *null*:

```
var pattn=new RegExp("[0-9]+");
document.write(pattn.exec("38 попугаев"));
```

В результате будет получено 38.

Если необходимо найти все соответствия, то при вызове конструктора *RegExp* следует указать дополнительный параметр “g”, указывающий на необходимость глобального поиска.

```
var pattn = new RegExp("[0-9]+", "g");
do {
  result  = pattn.exec("1 попугай, 2 попугая,…, 38 попугаев");
  document.write(" " + result);
} while (result != null)
```

В результате будет получено 1 2 38 null.

Метод *compile*() применяется для изменения ранее созданного шаблона.

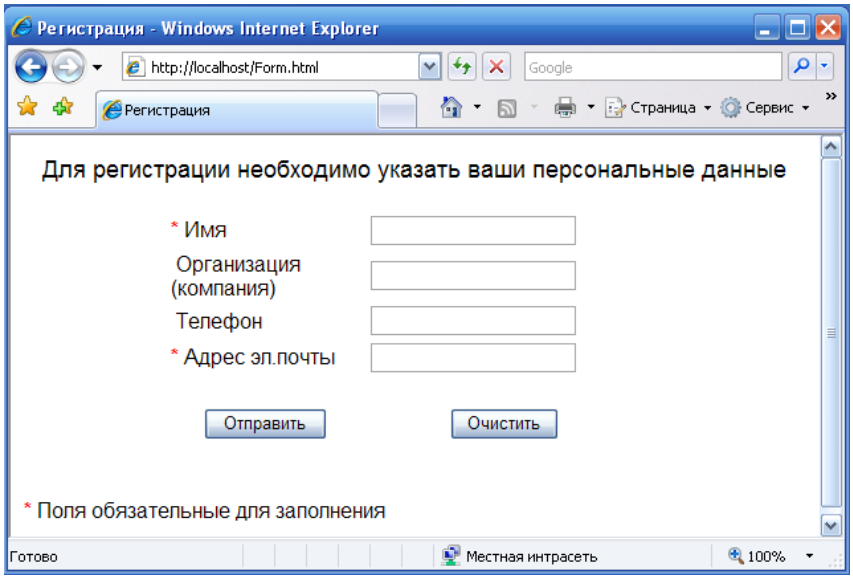
```
var pattn = new RegExp("[0-5]+");
document.write(pattn.exec("38 попугаев"));
pattn.compile("[6-9]+");
document.write(";" + pattn.exec("38 попугаев"));
```

В результате будет получено 3;8.

Общая постановка задачи

1. Разработать скрипт проверки значений, введенных пользователем в поля формы для регистрации.

Для выполнения задания необходимо создать веб-страницу, содержащую форму с полями, следующего вида:



В тэге <form> добавьте обработчик события отправки данных вида: onSubmit = "*CheckData*(); return false;". В данном случае указана функция обработчик *CheckData()*. Оператор *return false*; предотвращает автоматическую отправку данных после выполнения функции-обработчика. Отправка данных будет выполняться из обработчика.

Добавьте на странице секцию JavaScript кода, описывающего функцию-обработчик:

```
function CheckData() {
  var ans;
  ans = confirm("Вы уверены, что хотите отправить введенные данные ?");
  if (ans) submit();
}
```

Как это видно из кода, функция *CheckData()* в случае подтверждения со стороны пользователя самостоятельно вызывает метод *submit()* для передачи данных из формы.

Далее необходимо добавить проверку значений, введенных в поля формы пользователем. Прежде всего, необходимо убедиться в том, что заполнены все поля, обязательные для ввода. Для этого можно использовать проверку на равенство нулю длины строки (свойство *length*), являющейся значением узла дерева документа, соответствующего полю ввода, например: *document.getElementById("uname").value.length*.

Следующая проверка должна контролировать структуру и содержимое полей. Для этого можно использовать объект *RegExp*, например:

```
var validEMail, pattn;
pattn = new RegExp("^[\.\-_A-Za-z0-9]+?@[\.\-A-Za-z0-9]+?\.[A-Za-z0-9]{2,6}$");
validEMail = pattn.test(document.getElementById("email").value));
```

В данном фрагменте описана проверка структуры электронного адреса из поля формы с идентификатором "*email*". Для проверки был использован шаблон на основе регулярного выражения.

Самостоятельно постройте регулярное выражение, описывающее шаблон для проверки номера телефона, и внесите все необходимые изменения и дополнения в функцию CheckData().

2. Поместить форму и **JS**-код сценария на FTP-сервере. Проверить работу обработчика.

3. Составить электронный отчет о проделанной работе. На титульном листе отчета указать URL для запуска разработанной программы на выполнение.